

Register Machine (machine)

Tijdens renovaties van de Bock Casemates – een netwerk van ondergrondse vestingwerken in Luxemburg-Stad – is een oude schatkist opgegraven, verzegeld door een nogal eigenaardig mechanisme.

De inscriptie op het deksel noemt het een *registermachine*, en legt uit dat het 64 registers heeft genummerd van 0 tot en met 63, die elk een niet-negatieve integer kunnen bevatten. In het begin bevatten alle registers 0.

Een verzameling knoppen en schakelaars laat je een *programma* invoeren bestaande uit een reeks instructies. De eerste instructie die je invoert krijgt het label 0, de volgende 1, enzovoort. Er zijn drie soorten instructies:

- **INCREMENT** x, i — De waarde opgeslagen in register x wordt met 1 verhoogd, waarna de machine de instructie met label i verwerkt.
- **DECREMENT** x, i, j — Als de waarde opgeslagen in register x gelijk is aan 0, verwerkt de machine de instructie met label i . Anders wordt register x met 1 verlaagd, waarna de machine de instructie met label j verwerkt.
- **HALT** — De machine stopt met het verwerken van instructies.

De machine begint met verwerken vanaf de instructie met label 0. Zodra het stopt, opent het de kist als de waarde opgeslagen in register 0 precies N is.

Als het programma niet stopt na het verwerken van 50 000 000 instructies, of als het probeert een instructie te verwerken die niet bestaat, zal de machine volledig uitvallen en de kist voor altijd verzegelen.

Jouw taak is een programma in te voeren dat de kist opent. Omdat de machine gevoelig is, wil je dat de lengte van het programma zo kort mogelijk is, en zeker niet meer dan 100 instructies.

Implementatie

Je moet een enkel `.cpp` bronbestand inleveren.



Bij de bijlagen van deze opgave vind je een sjabloon `machine.cpp` en een voorbeeldgrader `grader.cpp` die je kunt gebruiken om je oplossingen lokaal te testen.

Je moet de volgende functie implementeren:

C++

```
void program_machine(int N)
```

- Het integer N stelt het getal voor dat opgeslagen moet worden in register 0 wanneer de machine stopt.

Je programma kan de volgende functies maximaal 100 keer aanroepen, die al gedefinieerd zijn in de grader:

C++

```
void add_increment(int x, int i)
```

C++

```
void add_decrement(int x, int i, int j)
```

C++

`void add_halt()`

Voorbeeld Grader

Bij de bijlagen van deze opgave vind je een vereenvoudigde versie van de grader die tijdens de beoordeling wordt gebruikt, die je kunt gebruiken om je oplossingen lokaal te testen. De voorbeeldgrader leest data uit `stdin`, roept de functie `program_machine` aan en schrijft terug naar `stdout` in het volgende formaat.

Het invoerbestand bestaat uit één regel met de integer N .

De uitvoer bestaat uit één regel:

- Als je programma de kist correct opent, is de uitvoer `OK L`, waarbij L de lengte van je programma is.
- Anders begint de uitvoer met `WA`, gevolgd door een korte reden (bijv. programma overschreed 100 instructies, programma stopte niet op tijd, enz.)

Randvoorwaarden

- $1 \leq N \leq 1\,000\,000$.

Scoring

Je programma wordt getest op meerdere testgevallen gegroepeerd in subtaken. Om de score van een subtaak te behalen, moet je programma de kist correct openen op elk testgeval van die subtaak.

Voor subtaak 3 wordt de score bepaald door de lengte van je programma. Laat L de maximale lengte van je programma zijn over alle testgevallen in deze subtaak. De score voor deze subtaak is $\min(77, 100 - L)$. Om de volledige 77 punten voor deze subtaak te behalen, moet je programma maximaal 23 instructies gebruiken op alle testgevallen van deze subtaak.

- **Subtask 0 [0 punten]:** Voorbeeldtestgevallen.
- **Subtask 1 [7 punten]:** $N \leq 99$.
- **Subtask 2 [16 punten]:** N is een macht van 2 (d.w.z. $N = 2^k$ voor een integer $k \geq 0$).
- **Subtask 3 [77 punten]:** Geen aanvullende beperking. (Let op: de score voor deze subtaak hangt af van de lengte van je programma.)

Voorbeelden

Grader

```
program_machine(3)
```

Oplossing

```
add_increment(1, 1)
add_increment(0, 2)
add_decrement(1, 4, 1)
add_halt()
add_increment(0, 3)
```

Grader

Oplossing

```
program_machine(5)
```

```
add_increment(0, 1)
add_increment(0, 2)
add_increment(0, 3)
add_increment(0, 4)
add_increment(0, 5)
add_halt()
```

Uitleg

In het eerste voorbeeld is $N = 3$. Het voorbeeld maakt een programma met 5 instructies:

0.	INCREMENT 1, 1
1.	INCREMENT 0, 2
2.	DECREMENT 1, 4, 1
3.	HALT
4.	INCREMENT 0, 3

De volgende tabel toont het label van elke instructie die de machine verwerkt en de inhoud van de registers 0 en 1 daarna:

Instructie	Register 0	Register 1
—	0	0
0	0	1
1	1	1
2	1	0
1	2	0
2	2	0
4	3	0
3	3	0

In het tweede voorbeeld is $N = 5$ en het getoonde voorbeeldprogramma gebruikt 6 instructies.